

**SIMATS ENGINEERING  
ASSESSMENT TOOL 1**

**COURSE NAME: Artificial Intelligence**

**COURSE CODE: CSA1709**

**CO1-AT1-Analytical Problem Solving**

|                        |                        |
|------------------------|------------------------|
| <b>Register Number</b> | <b>192212301</b>       |
| <b>Name</b>            | <b>k.Abhishek dass</b> |

**COURSE OUTCOME COVERED**

**CO1:** Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

*Assessment Tool Weightage: 40%*

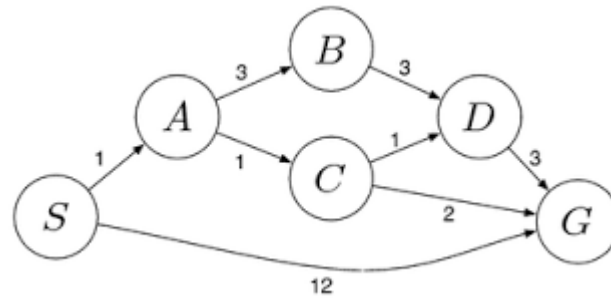
**QUESTIONS: 5**

**Total Marks: 50**

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
  - i. What are the percept's for this agent?
  - ii. Characterize the operating environment.
  - iii. What are the actions the agent can take?
  - iv. How can one evaluate the performance of the agent?
  - v. What sort of agent architecture do you think is most suitable for this agent? Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the

starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



**Code of Conduct:**

I \_\_\_\_\_ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

**Signature of the student:**

**RUBRICS FOR CALCULATION:**

| Criterion                 | Level 4<br>(Exemplary)                                                            | Level 3<br>(Proficient)                   | Level 2<br>(Developing)                    | Level 1<br>(Beginning)                        | Max<br>Marks |
|---------------------------|-----------------------------------------------------------------------------------|-------------------------------------------|--------------------------------------------|-----------------------------------------------|--------------|
| Problem Understanding     | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps       | Partial understanding; misses key elements | Misinterprets or unable to understand problem | 10           |
| Method Selection          | Selects most appropriate method/formula with strong justification                 | Selects correct method with minor errors  | Inappropriate or partially correct method  | Unable to select method                       | 10           |
| Solution Process          | Logical, systematic steps with correct approach throughout                        | Mostly correct steps with minor mistakes  | Disorganized steps; multiple errors        | No clear process                              | 12.5         |
| Accuracy of Calculation   | All calculations accurate; correct final answer                                   | Minor calculation errors                  | Frequent errors; incorrect final answer    | No valid calculations                         | 10           |
| Interpretation of Results | Clearly interprets results with units and engineering relevance                   | Basic interpretation with minor omissions | Limited or unclear interpretation          | No interpretation                             | 7.5          |

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring marker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Sol

## : The Alternative Solution

**1. Fill the 4-gallon jug:** Step 1.

Fill the **4-gallon jug** to the top from the pump.

- *State: 4-gal jug: 4 gallons | 3-gal jug: 0 gallons*

**2. Pour into the 3-gallon jug:** Step 2.

Pour water from the 4-gallon jug into the **3-gallon jug** until it is full. This leaves exactly 1 gallon in the larger jug.

- *State: 4-gal jug: 1 gallon | 3-gal jug: 3 gallons*

**3. Empty the 3-gallon jug:** Step 3.

Dump out the **3-gallon jug** onto the ground completely.

- *State: 4-gal jug: 1 gallon | 3-gal jug: 0 gallons*

**4. Transfer the remaining water:** Step 4.

Pour the **1 gallon** left in the 4-gallon jug into the empty **3-gallon jug**.

- *State: 4-gal jug: 0 gallons | 3-gal jug: 1 gallon*

**5. Refill the 4-gallon jug:** Step 5.

Fill the **4-gallon jug** to the top from the pump again.

- *State: 4-gal jug: 4 gallons | 3-gal jug: 1 gallon*

**6. Fill the remaining space:** Step 6.

Pour water from the 4-gallon jug into the **3-gallon jug** until the 3-gallon jug is full.

Since the 3-gallon jug already had 1 gallon in it, it only has room for **exactly 2 more gallons**. Pouring those 2 gallons out of your 4-gallon jug leaves you with **exactly 2 gallon** remaining inside it.

- *State: 4-gal jug: 2 gallons | 3-gal jug: 3 gallons*

6. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

- vi. What are the percept's for this agent?
- vii. Characterize the operating environment.
- viii. What are the actions the agent can take?
- ix. How can one evaluate the performance of the agent?
- x. What sort of agent architecture do you think is most suitable for this agent? Why?

Sol.

## I . What are the precepts for this agent?

Precepts are the sensory inputs the rover receives from its environment. For a Mars Rover, these include:

- **Visual Inputs:** Images from navigation cameras (Netcams), hazard avoidance cameras (Hazcams), and high-resolution science cameras.
- **Physical & Positional States:** Inertial Measurement Unit (IMU) data, accelerometers, gyroscopes, and wheel-rotation/slippage sensors.
- **Scientific Data:** Spectral readings (infrared, X-ray), laser-induced breakdown spectroscopy (LIBS) data, and microscopic imagery of rocks.

- **Environmental States:** Atmospheric temperature, pressure, wind velocity, and solar radiation levels.
- **System Status:** Battery charge levels, internal temperature of components, and memory storage capacity.
- **Communication Inputs:** Command signals received from Earth or relay orbiters.

## ii. Characterize the operating environment

The operating environment of Mars can be classified across standard AI environment dimensions:

- **Partially Observable:** The rover cannot perceive the entire planet or even what lies immediately behind a large boulder or beneath the surface without direct interaction.
- **Stochastic:** The environment is unpredictable. Dust storms can suddenly block solar panels, wind can shift sand, or wheels can slip unpredictably on loose soil.
- **Sequential:** Decisions are sequential. An action taken now (e.g., driving down a steep crater or draining battery power to drill a rock) directly dictates the options and safety of future states.
- **Dynamic:** The environment changes while the agent is deliberating (e.g., changing sunlight angles, temperature drops, wind, and moving dust).
- **Continuous:** States such as location coordinates, speed, steering angles, and temperature are continuous rather than discrete.
- **Single-agent:** While orbiters and other landers exist, the rover primarily operates alone in its immediate physical vicinity.

## iii. What are the actions the agent can take?

Actions are executed through the rover's actuators to interact with and alter its state or the environment:

- **Locomotion:** Steering, driving forward or backward, and adjusting individual wheel speeds.
- **Manipulation:** Moving and positioning the robotic arm (using joints/motors) to place instruments against targets.
- **Science Operations:** Drilling into rocks, scooping soil, brushing away dust, and firing lasers for chemical analysis.
- **Data Acquisition:** Pointing cameras and taking images/panoramas.
- **Communication:** Pointing the high-gain antenna towards Earth or passing orbiters to transmit telemetry and scientific findings.
- **Thermal/Power Control:** Activating internal heaters or positioning solar panels (if equipped) to maximize energy capture.

## iv. How can one evaluate the performance of the agent?

The performance measure quantifies how successfully the rover achieves its mission objectives:

- **Scientific Yield:** The volume, quality, and uniqueness of scientific data (images, chemical compositions, core samples) successfully gathered and transmitted to Earth.
- **Safety & Survivability:** Minimizing wear and tear on hardware (e.g., preventing wheel damage), avoiding hazardous terrain (not getting stuck in sand dunes), and maintaining safe thermal/power levels.
- **Resource Efficiency:** Maximizing distance travelled and science conducted per unit of electrical energy consumed.
- **Autonomy Success Rate:** The ratio of commands executed successfully without requiring human intervention or emergency overrides from Earth.

## v. What sort of agent architecture do you think is most suitable for this agent? Why?

The most suitable architecture is a **Hybrid Agent Architecture** (specifically combining a **Utility-Based / Goal-Based Agent** with a **Reactive Layer**).

Why?

1. **High Latency of Human Control:** Due to the massive distance between Earth and Mars, radio signals take anywhere from 4 to 24 minutes one way. A simple reflex agent is too primitive, while a purely centralized planner on Earth cannot react to real-time hazards.
2. **Goal-Based Planning:** The rover must be given high-level goals (e.g., *"Navigate to that outcrop and drill"*). It needs a deliberative system to plot a safe path, schedule tasks, and budget its limited battery power over the course of a Martian day (Sol).
3. **Reactive Safety:** At the same time, it needs a fast, reactive feedback loop. If a hazard camera detects a sudden drop-off or a wheel sensor detects extreme slippage, the rover must immediately halt or adjust its path without waiting for instructions from Earth.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

Sol.

## 1. Incremental Formulation (Recommended)

This method places one queen at a time, moving left to right from column 1 to column 8, ensuring that no queen is placed in an attacking position.

- **Initial State:** An empty  $8 \times 8$  chessboard.
- **State Space Representation:** An array or configuration indicating the positions of  $k$  queens on the board, where  $0 \leq k \leq 8$ , such that no two queens attack each other.
  - *Example representation:* A list of length  $k$ , where index represents the column and value represents the row: [row\_col1, row\_col2, ...].
- **Actions / Successor Function:** Place a queen in the leftmost empty column (column  $k + 1$ ) such that it is not attacked by any already placed queen.
- **Transition Model:** Returns the board with the new queen added to column  $k + 1$ .
- **Goal Test:** 8 queens are placed on the board, and none of them attack each other.
- **Path Cost:** 0 (we only care about finding the final configuration, not the cost of the path to reach it).

## 2. Complete-State Formulation

This method starts with all 8 queens on the board (e.g., one in each column) and iteratively repositions them to resolve conflicts (often used with Local Search strategies like Hill Climbing).

- **Initial State:** All 8 queens are placed on the board, typically with one queen per column in a random row.

- **State Space Representation:** A vector of length 8, where each index  $i$  corresponds to column  $i$ , and the value at that index corresponds to the row number (1 to 8).
  - *Size of state space:*  $8^8 = 16,777,216$  possible states.
- **Actions / Successor Function:** Move any single queen to another row in its same column.
- **Transition Model:** Returns the board configuration with the selected queen moved to its new row.
- **Goal Test:** 8 queens are on the board, and the number of attacking pairs equals zero.
- **Path Cost:** 0.

### 3. Comparing Search Space Sizes

Formulating the problem components intelligently dramatically alters how much of the search space needs to be explored:

1. **Naïve Formulation (Any square):**

If you just place 8 queens on any of the 64 squares, the state space is:

$$\binom{64}{8} \approx 4.4 \times 10^9 \text{ states}$$

2. **Complete-State Column-Restricted Formulation:**

If we enforce that exactly one queen lives in each column, the space drops to:

$$8^8 = 16,777,216 \text{ states}$$

3. **Incremental Formulation with Backtracking:**

By building the solution column by column and immediately abandoning paths where a new queen is under attack, the search tree is pruned down to only **2,057** states to examine in order to find all valid solutions.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

Sol.

### 1. Identify the Type of Problem-Solving Agent

For this scenario, a **Goal-Based, Goal-Formulating, and Planning Agent** (commonly referred to as a **Simple Problem-Solving Agent**) is the most suitable.

**Why?**

- **Goal-Formulation:** The agent starts by establishing a clear goal based on the customer's input: *to reach a specific destination*.
- **Problem-Formulation:** It defines the states (current location, intermediate routes, and target destination) and actions (booking, choosing cab types like mini, micro, sedan, shared, or prime).
- **Search and Planning:** The agent must search through a set of alternative paths, ride choices, pricing, and estimated time of arrivals (ETAs) to generate a sequence of actions (a plan) that satisfies the user's constraints (budget, comfort, speed).
- **Execution:** Once the plan (the selected cab and route) is decided, the agent executes the steps to guide the user to their destination.

### 2. Formulation of the Problem Components

Before writing the pseudocode, we formulate the problem components for the agent:

- **Initial State:** Customer at the pickup location with their preferences (comfort, budget).
- **Goal State:** Customer successfully arrived at the specified destination.
- **States:** Combinations of potential geographic locations (coordinates), available cabs nearby, and selected cab configurations.
- **Actions:**
  1. Input pickup and destination locations.
  2. Retrieve available cabs and pricing.
  3. Filter/select cab type based on comfort preference (mini, micro, sedan, shared, prime).
  4. Book cab.
  5. Travel along the computed route.
- **Path Cost:** A function of money (cab fare) and time (ETA), optimized according to the user's selected comfort level.

### 3. Pseudocode for the Problem-Solving Agent

Below is the standard artificial intelligence pseudocode for a **Simple Problem-Solving Agent** adapted specifically to this Ola Cab booking scenario.

```

                                input

# Update the state based on the latest percept (user input or current location updates)

# If the action sequence is empty, we need to formulate a goal, formulate a problem, and find a
plan
if is
    # 1. Goal Formulation: Define where the user wants to go

    # 2. Problem Formulation: Define states, actions, cost metrics (cab types: mini, micro,
    sedan, etc.)

    # 3. Search / Planning: Find the best sequence of choices (cab selection + route planning)

    if is
        return or with

    # 4. Execution: Remove the first action from the planned sequence and execute it

return

```

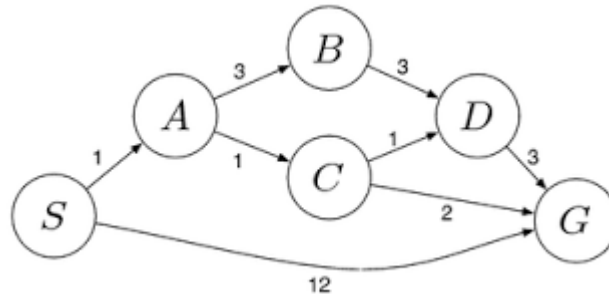
#### Detailed Execution of the Helper Functions:

- **FORMULATE-GOAL:** Takes the customer's destination input and comfort preference (e.g., "Reach *Point B* using a *Sedan*").
- **FORMULATE-PROBLEM:** Looks at available Ola drivers in the vicinity, filters them by the chosen class (e.g., *Sedan*), computes fares, and maps potential traffic routes.

- **SEARCH:** Calculates the optimal route and cab option that matches the user's criteria.
- **action:** Translates to execution steps such as Book Cab(Sedan), Navigate Route(Route A), and finally Complete Trip().

5.A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse **G** using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



## Step-by-Step Execution of UCS

### Step 1: Initialize

- **Priority Queue (PQ):** [(S, 0, Path: S)]
- **Explored Set:** {}

### Step 2: Expand S (Cost = 0)

- **Action:** Dequeue (S, 0). It is not the goal. Add S to Explored.
- **Neighbors of S:**
  - A via S → A: Cost = 0 + 1 = 1. Add to PQ.
  - G via S → G: Cost = 0 + 12 = 12. Add to PQ.
- **PQ state:** [(A, 1, Path: S->A), (G, 12, Path: S->G)]
- **Explored Set:** {S}

### Step 3: Expand A (Cost = 1)

- **Action:** Dequeue (A, 1) (lowest cost). It is not the goal. Add A to Explored.
- **Neighbors of A:**
  - C via A → C: Cost = 1 + 1 = 2. Add to PQ.
  - B via A → B: Cost = 1 + 3 = 4. Add to PQ.
- **PQ state:** [(C, 2, Path: S->A->C), (B, 4, Path: S->A->B), (G, 12, Path: S->G)]
- **Explored Set:** {S, A}

### Step 4: Expand C (Cost = 2)

- **Action:** Dequeue (C, 2) (lowest cost). It is not the goal. Add C to Explored.
- **Neighbors of C:**
  - D via C → D: Cost = 2 + 1 = 3. Add to PQ.
  - G via C → G: Cost = 2 + 2 = 4. Add to PQ.
- **PQ state:** [(D, 3, Path: S->A->C->D), (B, 4, Path: S->A->B), (G, 4, Path: S->A->C->G), (G, 12, Path: S->G)]
- **Explored Set:** {S, A, C}

### Step 5: Expand D (Cost = 3)

- **Action:** Dequeue (D, 3). It is not the goal. Add D to Explored.
- **Neighbors of D:**



- $G$  via  $D \rightarrow G$ : Cost =  $3 + 3 = 6$ . Add to PQ.
- **PQ state:** [(B, 4, Path: S->A->B), (G, 4, Path: S->A->C->G), (G, 6, Path: S->A->C->D->G), (G, 12, Path: S->G)]
- **Explored Set:** {S, A, C, D}

#### Step 6: Expand $B$ (Cost = 4)

- **Action:** Dequeue ( $B, 4$ ). It is not the goal. Add  $B$  to Explored.
- **Neighbors of  $B$ :**
  - $D$  via  $B \rightarrow D$ : Cost =  $4 + 3 = 7$ . (Can be ignored since  $D$  is already in the Explored Set).
- **PQ state:** [(G, 4, Path: S->A->C->G), (G, 6, Path: S->A->C->D->G), (G, 12, Path: S->G)]
- **Explored Set:** {S, A, C, D, B}

#### Step 7: Expand $G$ (Cost = 4)

- **Action:** Dequeue ( $G, 4$ ).
- **Goal Test: Success!** Since  $G$  is the goal, the search terminates here.

### Summary Table of UCS Iterations

| Iteration | Node Expanded | Current Cost | Queue State (Node, Cost, Path)                                              | Explored Set       |
|-----------|---------------|--------------|-----------------------------------------------------------------------------|--------------------|
| 0         | —             | —            | [(S, 0, S)]                                                                 | {}                 |
| 1         | S             | 0            | [(A, 1, S->A), (G, 12, S->G)]                                               | {S}                |
| 2         | A             | 1            | [(C, 2, S->A->C), (B, 4, S->A->B), (G, 12, S->G)]                           | {S, A}             |
| 3         | C             | 2            | [(D, 3, S->A->C->D), (B, 4, S->A->B), (G, 4, S->A->C->G), (G, 12, S->G)]    | {S, A, C}          |
| 4         | D             | 3            | [(B, 4, S->A->B), (G, 4, S->A->C->G), (G, 6, S->A->C->D->G), (G, 12, S->G)] | {S, A, C, D}       |
| 5         | B             | 4            | [(G, 4, S->A->C->G), (G, 6, S->A->C->D->G), (G, 12, S->G)]                  | {S, A, C, D, B}    |
| 6         | G             | 4            | <b>Goal Found! Search Terminates.</b>                                       | {S, A, C, D, B, G} |

### Final Result

- **Least-Cost Path:** S -> A -> C -> G
- **Minimum Total Cost:** 4 (Calculated as  $1 + 1 + 2 = 4$ )

**Code of Conduct:**

I K.Abhishek dass / 192212301 (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

*I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:**